Deep Learning: Making It Learnable

A course companion in native Python and PyTorch

Heman Shakeri

2026-07-07

Table of contents

Preface

> ⚠ Warning
>
> **Draft.** This book is being written in the open as the companion text for DS 6050 Deep Learning at UVA's School of Data Science. Chapters appear as they mature; everything here runs — every figure and result is produced by the code on the page.

Deep learning did not appear out of nowhere. Nearly every idea that powers modern AI is a classical idea — a line of best fit, a hand-designed image filter, a kernel-weighted average — that someone dared to ask one question about:

> *What if we made this* ***learnable****?*

That question is the spine of this book. We will ask it over and over:

- Take **linear regression**, let its features be learned, and you get the multilayer perceptron.
- Watch the MLP **fail to generalize** on images — see the failure in pictures — and the cure (respecting the structure of the data) becomes obvious: an **inductive bias**.
- Take the **fixed filters** of classical computer vision, make them learnable, and you get the convolutional network.
- Take **kernel regression** — a century-old way to average nearby evidence — make the similarity function learnable, and you get **attention**, then self-attention, then the Transformer.
- Then comes the **BERT moment**: stop training from scratch at all, and adapt what has already been learned.

Each part of the book replays this same move at a larger scale. By the end, the modern stack should feel less like a zoo of architectures and more like one idea, compounding.

How to read this book

Every chapter follows the course's learning loop: **read → run → check**. The code is native Python and PyTorch, written to run on an ordinary laptop CPU — small data, honest experiments. Watch the lecture videos, read the chapter, run the cells, and test yourself against the self-checks on the course site.

Acknowledgments

To the students of DS 6050, whose questions shaped every explanation here.

Part I.

I · From Lines to Networks

1. Linear Regression, the Mother Model

The core task of everything we will do in this course is to teach a computer to learn a function from examples. Before we can appreciate what is *deep* about deep learning, we need one complete, honest example of learning — a model, a loss, and a way to improve. Linear regression is that example. It is the smallest model that contains the entire supervised learning story, and by the end of this chapter we will have built it three times: once in closed form, once by gradient descent, and once the way you will build everything else in this book — with PyTorch modules. All three will agree, and you will know exactly why.

1.1. Learning from examples

We start with a dataset of examples,

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n},$$

where each $\mathbf{x}_i \in \mathbb{R}^d$ is an input with d features and y_i is the desired output — the *supervision signal*. Stack the inputs as rows and you get the data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$: n samples down, d features across, with the targets collected in a vector $\mathbf{y}$.

Our goal is a flexible function f such that

$$f(\mathbf{x}_i) \approx y_i \quad \text{and, crucially, } f \text{ generalizes to unseen data.}$$

That second clause is the whole game. We do not care how well f recites the training examples; we care how it behaves on a new $\mathbf{x}$ it has never seen — one drawn from the same distribution as the training data.

Supervised learning, three flavors

The range of y decides the task and, as we will see, the natural loss:

Task	Range of y	Typical loss
Regression	$\mathbb{R}$	Mean squared error
Classification	$\{0, \dots, C-1\}$	Cross-entropy
Structured prediction	sequences, images, graphs	task-specific

This chapter is regression; Chapter 2 handles classification with the same machinery.

1.1.1. Why is this hard?

This task sounds simple, but it is fundamentally hard: for any finite set of examples, there are *infinitely many* functions that fit them perfectly. We want the one that is most suited to our problem — so the learning algorithm needs a nudge in the right direction. That guiding assumption is called its **inductive bias**, and it is a thread we will pull on for the rest of the book.

- A **linear model** has a *strong* bias: it assumes the world is linear. Simple, stable — and systematically wrong when the data is not linear (high bias, low variance).
- A **deep neural network** has a *weak* bias. Its flexibility lets it learn complex patterns, but it can memorize the training data instead of the underlying rule (low bias, high variance). Some networks can memorize an entire training set — and then perform poorly the day you deploy them.

The key to modern deep learning is to use a flexible model and fight overfitting with data, regularization, and — the deepest idea of all — inductive biases matched to the task. That last idea gets its own chapter (Chapter 6).

To make any of this concrete, we parameterize a family of functions $f_{\mathbf{w}}$ by a set of tuning parameters $\mathbf{w}$ — think of each parameter as a knob. *Learning* is finding a good setting $\hat{\mathbf{w}}$ of the knobs; *inference* is using $f_{\hat{\mathbf{w}}}$ to predict on unseen data. That is the vocabulary we will use all course.

1.2. The linear model and its geometry

Linear regression assumes a linear relationship:

$$y = \mathbf{w}^\top \mathbf{x} + b.$$

Geometrically, $\mathbf{w}$ sets the orientation — the slope — of a hyperplane, and the bias b shifts it away from the origin. In two dimensions this is the familiar line through a scatter of points; in d dimensions it is a d-dimensional plane, but nothing about the algebra changes.

1.2.1. The augmentation trick

Carrying b separately clutters the algebra, so we will often fold it into the weights: append a column of ones to the data matrix, making $\mathbf{X} \in \mathbb{R}^{n\times(d+1)}$, and append b to the weight vector, making $\mathbf{w} \in \mathbb{R}^{d+1}$. Then

$$y = \mathbf{w}^\top \mathbf{x} \qquad \text{(bias included)}.$$

Whenever you see a linear model written without a bias — here or inside a neural network — the bias has not disappeared; it is living inside $\mathbf{w}$.

1.3. The loss: what makes a hyperplane "bad"?

To find the best hyperplane we must first quantify error. The standard choice for regression is the **mean squared error (MSE)**:

$$\mathcal{L}(\mathbf{w}) \;=\; \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2 \;=\; \frac{1}{n}\,\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2\,. \tag{1.1}$$

The loss is the guiding principle of training: it is how we tell the model *you are a bad model, and here is by how much* — and the model has to find a way to improve. The choice of loss is not arbitrary, and we will justify this one honestly at the end of the chapter.

> The first "make it learnable" seed
>
> Look closely at $y = \mathbf{w}^\top \mathbf{x} + b$. A single neuron in a neural network computes *exactly this*, followed by a nonlinear squashing function $\sigma(\mathbf{w}^\top \mathbf{x} + b)$. Linear regression **is** a neuron with the identity activation. Everything we learn here — loss, gradients, optimization — transfers directly. The rest of this book is, in a precise sense, the story of what happens when we take this fixed, simple recipe and progressively let more of it be *learned*.

1.4. Finding the best weights, method 1: solve it exactly

For this one special problem we can find the exact minimizer. Set the gradient of Equation 1.1 to zero and you get the **normal equations**:

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}. \tag{1.2}$$

There is a picture behind this formula worth carrying for the rest of your career. Our predictions $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ can only ever live in the **column space** of $\mathbf{X}$ — the span of its feature columns. If the true $\mathbf{y}$ lies outside that space (it almost always does), the best we can do is **project** $\mathbf{y}$ onto it. The leftover error $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ is what our model cannot explain, and at the optimum it is *perpendicular* to the column space — no adjustment of the knobs can reduce it further.

Elegant — so why is this formula almost absent from deep learning? Two reasons. Solving it costs $O(d^3)$, which is hopeless when d runs to millions or billions. And it only exists because the model is linear; the moment we add a nonlinearity, there is no closed form to solve. Our datasets are large and our models are not linear — we have neither luxury. We need another way.

1.5. Finding the best weights, method 2: walk downhill

Here is the geometric intuition. Picture the loss as a landscape over the parameter space, and imagine standing on it *blindfolded*, trying to find the lowest valley. You cannot see the valley —

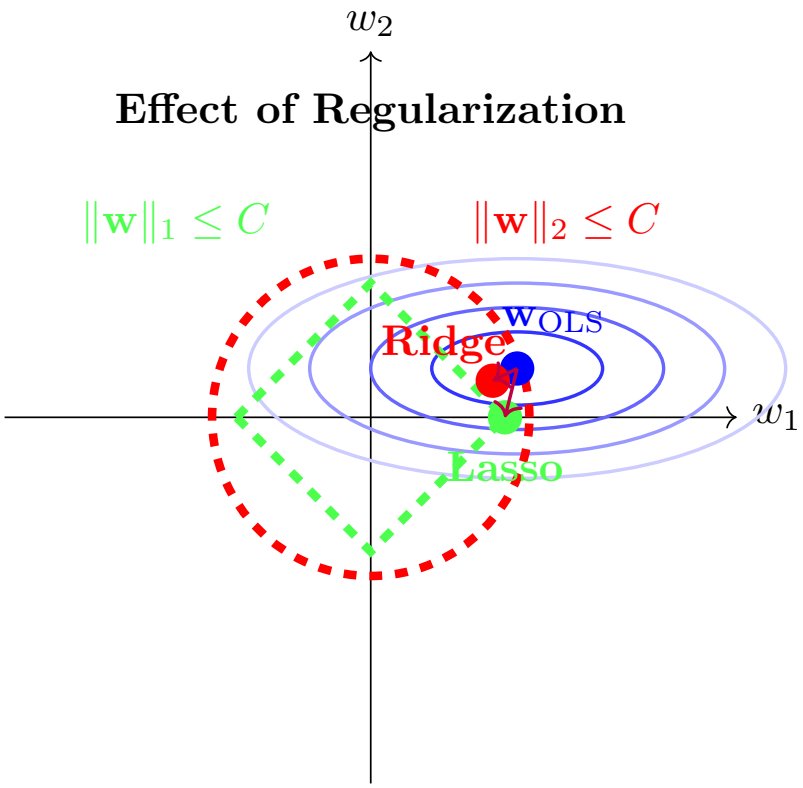

Figure 1.1.: The normal equations, geometrically: the optimal prediction $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$ is the projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$; the residual $\mathbf{e}$ is orthogonal to it.

but you can feel the slope under your feet. So you feel the slope, take a small step downhill, and repeat.

The mathematical form of this hill-descending intuition is **gradient descent**:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \, \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(t)}), \tag{1.3}$$

where the *learning rate* η sets the step size. For the MSE loss the gradient has a clean closed form — with $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{w} \in \mathbb{R}^d$, and $\mathbf{y} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \ \in \ \mathbb{R}^d. \tag{1.4}$$

This iterative loop — predict, measure the loss, feel the gradient, step — is exactly what modern deep learning frameworks automate, at billion-parameter scale. When the dataset itself is huge we will not even use all of it per step: a small random *batch* gives a good-enough gradient. That refinement (stochastic gradient descent) matters enough to get its own treatment in Chapter 4.

1.6. Build it three times

Enough theory — let us implement linear regression, and let us do it in PyTorch. You have already built this in NumPy in earlier courses, so we will use it as an excuse to get comfortable with tensors, while still doing everything from scratch.

> **Coding hygiene**
>
> Two habits worth forming from the very first cell: **fix your seeds** so runs are reproducible, and **annotate your functions with types** — not critical for Python to run, but it makes code readable and debugging far easier.

```
import torch
import matplotlib.pyplot as plt

torch.manual_seed(6050)  # reproducible runs, always
```

```
<torch._C.Generator at 0x10fe8c3d0>
```

First, synthetic data. We control the ground truth — the true weights and bias — so we can check whether each method actually recovers them.

```python
def make_synthetic_data(
    weights: torch.Tensor,   # (d,) true weights
    bias: float,             # true bias
    n_samples: int,
    noise: float = 0.1,      # std of Gaussian noise
) -> tuple[torch.Tensor, torch.Tensor]:
    """y = X w + b + eps, with eps ~ N(0, noise^2)."""
    d = weights.shape[0]
    X = torch.randn(n_samples, d)              # (n, d) feature matrix
    y = X @ weights + bias                     # (n,)  clean targets
    y += noise * torch.randn(n_samples)        # add irreducible noise
    return X, y

true_w = torch.tensor([2.0, -3.4])
true_b = 4.2
X, y = make_synthetic_data(true_w, true_b, n_samples=200)
X.shape, y.shape   # (200, 2), (200,) - always check your shapes
```

```
(torch.Size([200, 2]), torch.Size([200]))
```

1.6.1. Take 1: the closed form

The normal equations, Equation 1.2, on the *augmented* matrix (the column of ones carries the bias):

```python
X_aug = torch.cat([X, torch.ones(X.shape[0], 1)], dim=1)   # (n, d+1)
w_ols = torch.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)   # solve, don't invert
print(f"closed form:      w = {w_ols[:2].numpy().round(3)},  b = {w_ols[2]:.3f}")
print(f"ground truth:     w = {true_w.numpy()},  b = {true_b}")
```

```
closed form:      w = [ 2.01  -3.402],  b = 4.196
ground truth:     w = [ 2.  -3.4],  b = 4.2
```

Two lines, and we recover the truth up to the noise floor. Remember what this cost us: $O(d^3)$, and the special grace of a linear model. Now the method that scales.

1.6.2. Take 2: gradient descent, from scratch

We write the model as a small class — a forward pass, a loss, manually derived gradients (Equation 1.4), and an update step. No autograd yet: we want to feel the mechanics once with our own hands.

```python
class LinearRegressionScratch:
    """Linear regression with manually derived gradients."""

    def __init__(self, input_size: int, lr: float = 0.1):
        self.w = 0.01 * torch.randn(input_size)   # small random start
        self.b = torch.zeros(1)
        self.lr = lr

    def forward(self, X: torch.Tensor) -> torch.Tensor:
        return X @ self.w + self.b                # (n,)

    @staticmethod
    def loss(y_hat: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        return ((y_hat - y) ** 2).mean()

    def step(self, X: torch.Tensor, y: torch.Tensor) -> float:
        y_hat = self.forward(X)                   # 1. predict
        loss = self.loss(y_hat, y)                # 2. measure
        err = y_hat - y                           # (n,) residuals
        grad_w = 2.0 * X.T @ err / len(y)         # 3. feel the slope  (d,)
        grad_b = 2.0 * err.mean()                 #    (bias grad = mean error)
        self.w -= self.lr * grad_w                # 4. step downhill
        self.b -= self.lr * grad_b
        return float(loss)

model = LinearRegressionScratch(input_size=2)
losses = [model.step(X, y) for _ in range(100)]
print(f"gradient descent: w = {model.w.numpy().round(3)},  b = {model.b.item():.3f}")
```

```
gradient descent: w = [ 2.01  -3.402],  b = 4.196
```

```python
plt.figure(figsize=(5.5, 3.2))
plt.plot(losses)
plt.xlabel("step")
plt.ylabel("MSE loss")
plt.yscale("log")
plt.tight_layout()
plt.show()
```

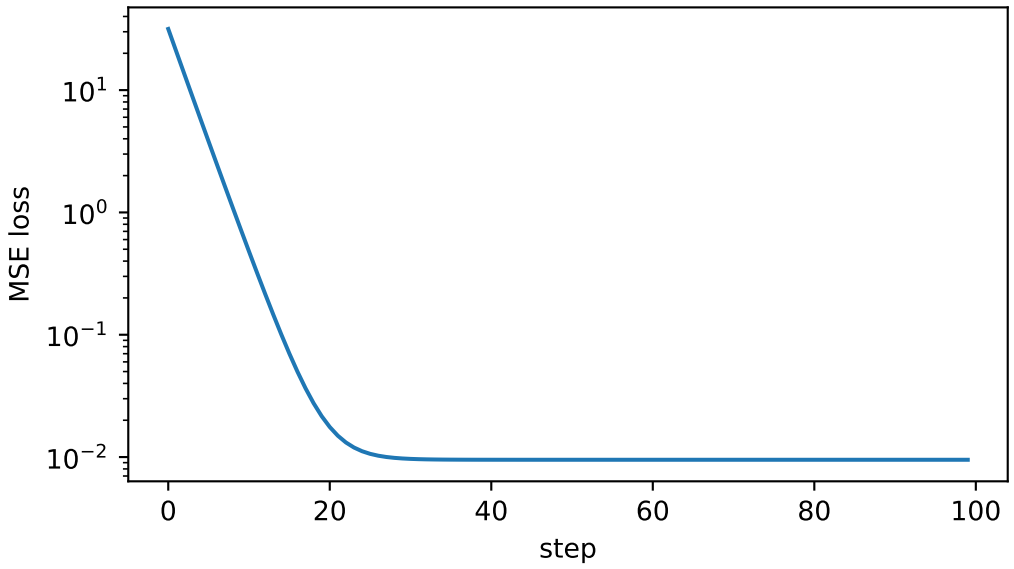

Figure 1.2.: The loss falls fast at first — steep slope underfoot — then flattens as we approach the valley floor set by the irreducible noise.

1.6.3. Take 3: the framework way

Finally, the idiom you will use for every model from here on: `nn.Linear` holds the knobs, autograd feels the slope for us, and the optimizer takes the step.

```
from torch import nn

net = nn.Linear(in_features=2, out_features=1)
optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

for _ in range(100):
    optimizer.zero_grad()               # clear old gradients - they accumulate!
    y_hat = net(X).squeeze(-1)          # (n, 1) -> (n)
    loss = loss_fn(y_hat, y)
    loss.backward()                     # autograd computes the gradient for us
    optimizer.step()

w_nn = net.weight.detach().squeeze()
b_nn = net.bias.item()
print(f"nn.Linear:        w = {w_nn.numpy().round(3)},  b = {b_nn:.3f}")
```

```
nn.Linear:        w = [ 2.01  -3.402],  b = 4.196
```

Three implementations, one answer:

```
grid = torch.linspace(X[:, 0].min(), X[:, 0].max(), 50)
line = lambda w, b: w[0] * grid + w[1] * X[:, 1].mean() + b

plt.figure(figsize=(5.5, 3.6))
plt.scatter(X[:, 0], y, s=12, alpha=0.4, label="data")
plt.plot(grid, line(w_ols, w_ols[2]), lw=3, label="closed form")
plt.plot(grid, line(model.w, model.b.item()), "--", lw=2, label="gradient descent")
plt.plot(grid, line(w_nn, b_nn), ":", lw=2, label="nn.Linear")
plt.xlabel("$x_1$"); plt.ylabel("$y$"); plt.legend()
plt.tight_layout()
plt.show()
```

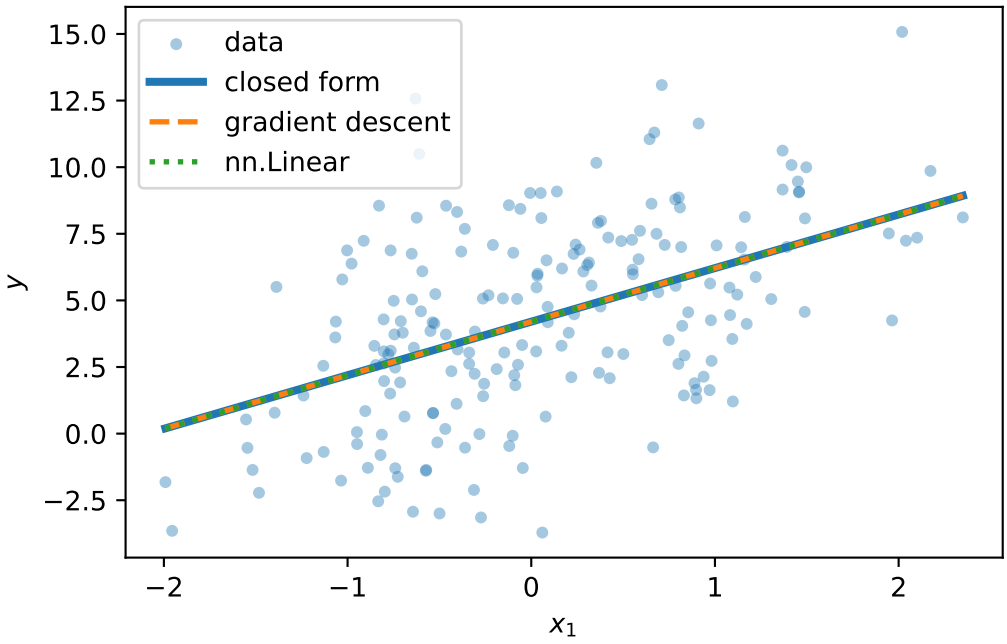

Figure 1.3.: All three methods find the same line (shown against feature x_1, second feature at its mean).

1.7. Why squared error? The maximum-likelihood view

The MSE is not an arbitrary choice. Assume the data really is generated by a linear process plus Gaussian noise:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

Then the probability of seeing output y given input $\mathbf{x}$ is a Gaussian centered on the model's prediction. **Maximum likelihood estimation** asks: which parameters make the observed data

most likely? Take the log of the likelihood over all n samples and the Gaussian's exponent comes down:

$$\log \mathcal{L}(\mathbf{w}, b) = C - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^\top \mathbf{x}_i + b)\right)^2.$$

Maximizing the log-likelihood is *exactly* minimizing the sum of squared errors. The loss encodes an assumption about the noise. This is a pattern to remember: when we meet cross-entropy in Chapter 2, it will be the same story with a different distribution.

1.8. A first look at bias and variance

Our learned $\hat{\mathbf{w}}$ depends on the training data — and the training data is a random sample. Collect a fresh dataset and you get a different $\hat{\mathbf{w}}$. So the model's expected error on a new point decomposes into three parts:

$$\text{Err} = \underbrace{\left(\mathbb{E}[\hat{y}] - y\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[(\hat{y} - \mathbb{E}[\hat{y}])^2\right]}_{\text{Variance}} + \underbrace{\sigma_\varepsilon^2}_{\text{Irreducible noise}}. \tag{1.5}$$

Bias measures how far the *average* prediction is from the truth, because of limiting assumptions like linearity — a simple model on complex data is systematically wrong (underfitting). **Variance** measures how much the prediction would change if we collected a fresh training set — the model's sensitivity to sampling noise; a complex model can fit the noise itself (overfitting). The **irreducible noise** is inherent to the data-generating process and cannot be learned away.

This is why we split data into training, validation, and test: the model sees the training set, the validation set referees the bias–variance trade-off, and the test set stays untouched until the end.

> ⚠ The U-shaped curve is a cartoon, not a law of nature
>
> The textbook picture — validation error falling then rising as complexity grows — is a helpful first mental model, and you should have it. But bias and variance are not a mechanical see-saw. More data shrinks variance (at a rate of roughly $1/n$, while leaving bias untouched); regularization, and inductive biases matched to the task, can buy capacity without exploding variance. Modern over-parameterized networks live in regimes where this cartoon bends in surprising ways — we will look at those pictures properly in Chapter 6.

1.8.1. Regularization, briefly

One lever we can pull right now: penalize complexity in the loss itself. **Ridge regression** adds an L_2 penalty,

$$\mathcal{L}_\lambda(\mathbf{w}) = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2, \qquad \hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top\mathbf{X} + \lambda I)^{-1}\mathbf{X}^\top\mathbf{y},$$

nudging the model toward smaller, more stable weights — a little more bias for a lot less variance. Its cousin **lasso** uses the L_1 norm, $\lambda\|\mathbf{w}\|_1$, and does something qualitatively different: it drives the least informative weights to *exactly zero*, performing automatic feature selection.

Let us see that difference, not just assert it. We build a problem designed to punish an unregularized model: 20 features of which only 5 matter, noisy targets, and — the cruel part — barely more samples than parameters. This is exactly the high-dimensional regime where regularization shines:

```
from sklearn.linear_model import Lasso   # coordinate descent for L1 (no closed form)

n, d = 25, 20                            # barely more samples than knobs!
w_true = torch.zeros(d)
w_true[:5] = torch.tensor([3.0, -2.0, 1.5, 2.5, -1.0])   # only 5 real features
Xh, yh = make_synthetic_data(w_true, bias=0.0, n_samples=n, noise=2.0)

# OLS and ridge in closed form
w_ols_h = torch.linalg.solve(Xh.T @ Xh, Xh.T @ yh)
lam = 10.0
w_ridge = torch.linalg.solve(Xh.T @ Xh + lam * torch.eye(d), Xh.T @ yh)
w_lasso = torch.tensor(
    Lasso(alpha=0.4).fit(Xh.numpy(), yh.numpy()).coef_, dtype=torch.float32
)

def report(name: str, w_hat: torch.Tensor) -> None:
    err = ((w_hat - w_true) ** 2).mean()
    zeros = int((w_hat.abs() < 1e-3).sum())
    print(f"{name:>6}:  weight MSE = {err:.4f}   near-zero weights = {zeros}/20")

report("OLS", w_ols_h)
report("ridge", w_ridge)
report("lasso", w_lasso)
```

```
   OLS:  weight MSE = 1.2347   near-zero weights = 0/20
 ridge:  weight MSE = 0.4400   near-zero weights = 0/20
 lasso:  weight MSE = 0.3489   near-zero weights = 12/20
```

OLS, with all its freedom and almost no data to discipline it, assigns large, confident, *wrong* weights to the 15 noise features — pure variance. Ridge shrinks everything and cuts the damage several-fold, but keeps every feature alive. Lasso does the one thing the others cannot: it identifies the noise features and sets them *exactly* to zero — a sparse, more interpretable model that has effectively performed feature selection for us. (Rerun this cell with `n = 100` and watch OLS become respectable again: variance decays like $1/n$.)

1.9. Okay, so — what did we just build?

We taught a computer a function from examples, completely:

1. **A model family** — hyperplanes, parameterized by knobs $\mathbf{w}, b$ (with the augmentation trick to fold b into $\mathbf{w}$).
2. **A loss** — MSE, which is maximum likelihood under Gaussian noise; the loss is how we tell the model how bad it is.
3. **An optimizer** — the exact projection when linearity permits, and gradient descent (feel the slope, step downhill) when it does not — which is always, from here on.
4. **The generalization lens** — bias vs. variance, the data splits that referee them, and regularization as a first counter-measure to overfitting.

And one reframing that will carry the whole book: linear regression is a single neuron with the identity activation,

$$\underbrace{y = \mathbf{w}^\top \mathbf{x} + b}_{\text{this chapter}} \quad \xrightarrow{\text{add a nonlinearity}} \quad \underbrace{y = \sigma(\mathbf{w}^\top \mathbf{x} + b)}_{\text{a neuron}}.$$

First, in Chapter 2, that squashing function turns our regressor into a classifier. Then, in Chapter 3, we stack neurons — and discover why the nonlinearity is not optional.

Exercises

1. **(Pencil.)** Derive the normal equations Equation 1.2 by expanding $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$, taking the gradient with respect to $\mathbf{w}$, and setting it to zero. Where exactly does the assumption that $\mathbf{X}^\top \mathbf{X}$ is invertible enter?
2. **(Pencil.)** Repeat the derivation with the ridge penalty $\lambda \|\mathbf{w}\|_2^2$ and show the solution becomes $(\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y}$. Why does the λI term guarantee invertibility for any $\lambda > 0$?
3. **(Code.)** In `make_synthetic_data`, raise `noise` to 1.0 and rerun all three implementations 10 times with different seeds. How much do the learned weights vary across runs? Which term of Equation 1.5 are you watching?
4. **(Code.)** Set the learning rate in `LinearRegressionScratch` to 1.0 and rerun. Describe what the loss curve does, and explain it using the blindfolded-descent picture.
5. **(Code.)** In the ridge/lasso experiment, sweep $\lambda \in \{0.01, 0.1, 1, 10, 100\}$ for ridge and plot weight MSE against λ. Where is the sweet spot, and what happens at the extremes? Connect your answer to bias and variance.

2. Linear Models that Classify: Logistic and Softmax

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The same linear machinery, pushed through a squashing function, classifies. Sigmoid inverts the logit; softmax turns scores into a distribution; cross-entropy scores it. All from maximum likelihood. Plant deliberately: softmax is the book's universal "scores → normalized weights" device — the exact same normalizer that will turn attention scores into attention weights in Part IV.

3. Nonlinearity and the MLP

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Stacked linear layers collapse back to one line — XOR proves it. One bend (ReLU) changes everything: neurons carve space into polytopes, depth multiplies expressivity, and the universal approximation theorem tells us what's possible (and what it doesn't promise).

4. Training: Loss and Stochastic Gradient Descent

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Where losses come from (maximum likelihood), why noisy gradients are a feature, and how momentum and Adam reshape the descent. An honest optimizer race on a problem where we know the truth.

5. Backpropagation

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The chain rule, organized. We draw the computation graph, push blame backwards by hand, build an 80-line autograd engine, and then let `torch.autograd` *do it — verifying it agrees with us.*

6. When It Fails: Generalization in Pictures, and Inductive Bias

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The signature chapter. We watch an MLP that aces MNIST fall apart when digits shift two pixels — in pictures. Capacity curves, sample complexity, the curse of dimensionality; and the cure: build what you know about the data into the architecture. That idea powers every remaining chapter.

Part II.

II · Vision: Learning the Filters

7. Filters and Convolution (Fixed Kernels)

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Before learning, engineering: Sobel edges, blurs, sharpeners. Convolution as structured, weight-shared linear maps — translation equivariance for free. Open with the Chapter 1 callback: a filter response is the dot-product similarity score of Chapter 1, slid across the image — the template is just still hand-made.

8. CNNs: Making the Filters Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Ask the question: what if the kernel entries were parameters? LeNet from scratch, padding/stride/pooling, receptive fields, and why a small CNN beats a big MLP on images.

9. Modern CNNs and Transfer Learning

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

VGG's stacked 3×3s, 1×1 convolutions, Inception's parallel paths, BatchNorm, ResNet's identity highway — and the practical superpower: start from a pretrained backbone.

Part III.

III · Sequences

10. Sequences and Recurrence

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Order matters. Parameter sharing across time gives the RNN; products of Jacobians give vanishing gradients; gates (LSTM/GRU) give memory a highway.

11. Encoder–Decoder, Teacher Forcing, Beam Search

> **Warning**
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Compress a sentence into a vector, then unroll it in another language. Teacher forcing, scheduled sampling, beam search — and the fixed bottleneck that sets up everything in Part IV.

Part IV.

IV · Attention: Learning the Similarity

12. Kernel Regression: Attention Before It Was Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

A century-old idea: predict by averaging nearby evidence, weighted by similarity. Nadaraya–Watson in NumPy, bandwidths, and the observation that this is already attention — with a fixed similarity function. Open with the two Chapter 1 callbacks (prediction as a weighted combination of training targets via the projection view, and the dot product as similarity score) plus the Chapter 2 callback (softmax normalizes scores into weights).

13. Attention: Making the Kernel Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Replace the fixed kernel with learned projections: queries, keys, values. Additive vs scaled dot-product (and why GPUs prefer the latter), the $\sqrt{d}$ scaling, masking, multi-head. The seq2seq bottleneck falls.

14. Self-Attention and the Transformer

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Let a sequence attend to itself and recurrence becomes optional. Positional encodings as rotations, multi-head subspaces, LayerNorm and residuals, the FFN as associative memory — the full block, built and trained.

15. The BERT Moment: Pretraining as the New Regime

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Stop training from scratch. Masked language modeling, bidirectional context, fine-tuning — and why 2018 changed the default workflow of the field.

16. Vision Transformers and Scaling Laws

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Patches as tokens: the transformer eats images, needs more data (inductive bias strikes again — this time as a trade), and obeys power laws. Chinchilla's 20-tokens-per-parameter arithmetic.

Part V.

V · The Pretrained Era

17. Adapting Pretrained Models: Prompting, PEFT, Quantization

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

When the model is too big to fine-tune, get surgical: prompting and in-context learning, LoRA's low-rank updates, quantization and QLoRA.

18. Alignment and RL Fine-Tuning

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

From next-token prediction to helpfulness: instruction tuning, preference learning, and RLHF in outline.

19. Generative Models: From PCA to Diffusion

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

One more make-it-learnable arc: PCA → autoencoder → VAE → diffusion (and GANs as the adversarial detour). Latents, ELBO, noise schedules.

A. Linear Algebra and the SVD

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The geometry the book leans on: matrices as transformations, eigenvectors, and the SVD as the master decomposition.

B. Tensors in Practice

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Shapes, broadcasting, indexing, and the layer-algebra habits that prevent 90% of PyTorch bugs.

C. Notation

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Symbols used throughout the book.